

Bloco 4.3, Propostas: tela de listagem — status de entrega

Data: 2026-09-04. Branch: feature-dev-breno-automacao. Working tree (sem commit, conforme a regra de só commitar sob pedido).

O que o bloco é

Tirar `/propostas` do `<EmConstrucao />` e entregar a listagem no mesmo padrão de `/orcamentos` e `/fd`: busca, filtros, paginação e estado inteiro na URL. A base veio do bloco 4.2 (tipos, helpers puros e guard de perfil), então aqui não houve regra nova de negócio — o trabalho foi montar a tela sobre o que já existia e resolver a única diferença estrutural de Propostas, que é o cliente não estar na tabela.

Atividade 1: colunas

Concluída. `propostas-table.tsx` sobre o `DataTable` genérico, sete colunas na ordem pedida: Número, Obra, Cliente, Data emissão, Validade, Valor final, Status.

Duas notas:

- **Obra sai como** `codigo_obra – nome`, um campo só, igual ao label do select de obra em `/fd`. Duas colunas separadas gastariam largura numa tabela que já tem sete.
- **Cliente vem de** `p.obra?.cliente?.nome`, dois níveis, porque o JOIN é aninhado (ver atividade 2). Data, validade e valor usam `formatDate` e `formatCurrency` com `tabular-nums`, como nas outras listagens.

Atividade 2: busca por número, obra e cliente

Concluída, e é o ponto que exigiu decisão.

Propostas **não tem** `cliente_id` — o cliente vem por obra (`propostas → obras → clientes`), como o bloco 4.2 já havia registrado em `PropostaListItem`. Consequência prática: não existe um `.or()` que alcance o nome do cliente, porque PostgreSQL não filtra por coluna de tabela aninhada dentro de um `or`. A busca resolve os IDs antes, no mesmo espírito do que `/orcamentos` faz com cliente, com uma diferença:

O passo de obra é feito em memória. A página já carrega a lista completa de obras para alimentar o select do filtro. Reusar essa lista para o match por nome e código da obra economiza uma consulta — sobra só a de `clientes`, que não tem como ser evitada. O helper é puro e testado:

```
obraIdsDaBusca(obras, busca, clienteIds): string[]
```

O `.or()` final da query fica com dois termos, `numero.ilike` e `obra_id.in(...)`, e o segundo é omitido quando nenhuma obra casa — igual ao tratamento de `clienteFilter` em `/orcamentos`.

Feito além do checklist: a busca também casa `codigo_obra`. O checklist pede nome da obra, mas quem digita "OB-2026" espera achar, e é a mesma linha de código.

Correção de robustez: `sanitizeBusca` remove vírgula, parêntese e aspas do termo antes de montar o `.or()`. O `.or()` do PostgREST recebe os filtros como uma string separada por vírgula, com os valores de `in(...)` entre parênteses, então um desses caracteres digitado na busca não devolve "nada encontrado" — ele quebra o parser e **derruba a query inteira**. As listagens de orçamentos e FD tinham o mesmo buraco e passaram a usar o helper compartilhado.

Atividade 3: filtros de obra, status e período

Concluída. `filters.tsx` client component, três selects mais o campo de busca com debounce de 300 ms, cada mudança escrevendo na URL e zerando `page` — o mesmo componente de `/fd`, adaptado.

- **Obra:** select alimentado pelo servidor via prop `obraOptions`, valor é o `id`, filtro vira `eq('obra_id', ...)`.
- **Status:** as quatro opções vêm de `STATUS_PROPOSTA_OPTIONS`, do 4.2, em vez de uma lista literal como em orçamentos. A querystring passa por `isPropostaStatus` antes de virar `eq`, então `?status=qualquercoisa` é ignorado em vez de retornar erro do banco.
- **Período de emissão:** 30d, 90d e ano, sobre `data_emissao`.

Vencidas entrou como quinta opção do select de status, resolvendo a especificação da pendência 3 do bloco 4.2. Como vencida **não é status do banco**, ela não pode viajar em `?status=`: a URL usa `?vencidas=1` e a query filtra `status = 'enviada'` com `data_validade < hoje`.

A decisão de UI que vale registro: **vencidas ocupa uma linha do mesmo select de status em vez de um controle próprio**. Um checkbox separado permitiria `?status=aprovada&vencidas=1`, combinação contraditória que sempre retornaria lista vazia. No mesmo select, um valor apaga o outro por construção — o handler escreve um e deleta o outro, e o servidor trata `vencidas` antes de `status`.

Atividade 4: paginação de 20

Concluída. `PAGE_SIZE = 20`, `.range(from, to)` com `count: 'exact'` e o componente `Pagination` já existente, reusado sem alteração. Os cinco filtros seguem em `extraParams`, então navegar entre páginas não perde estado.

Ordenação: `data_emissao` desc, com dois cuidados que as outras listagens não precisam ter. `data_emissao` é **nullable** em propostas, então vai `nullsFirst: false` — rascunho sem data de emissão fica no fim, não no topo. E existe um segundo `order` por `numero` desc como desempate: sem ele, propostas com a mesma data de emissão podem sair em ordem diferente entre duas requisições, e um registro repetiria na página 2 enquanto outro desapareceria.

Atividade 5: badge de status

Concluída, já estava pronta. As quatro chaves de proposta entraram no `StatusBadge` no bloco 4.2, com exatamente as cores pedidas: rascunho cinza, enviada azul, aprovada verde, rejeitada vermelho. Nada a fazer no componente.

O que a listagem acrescentou foi o **selo âmbar "Vencida"** ao lado do badge, quando `isPropostaVencida` é verdadeiro — a outra metade da pendência 3 do 4.2. Ele é um selo separado, não uma cor do `StatusBadge`, justamente porque não é status: uma proposta vencida continua `enviada` no banco.

Atividade 6: EmptyState e botão Nova proposta

Concluída. `canCreate` é admin ou comercial, checado por `getCurrentProfile`, e governa tanto o botão do topo quanto o CTA do `EmptyState`. Ícone `FileSignature`, na linha do `FileText` de orçamentos e do `HandCoins` de FD.

Os dois estados vazios são distintos, como nas outras listagens: sem nenhum registro aparece o `EmptyState` com o CTA; com filtro aplicado e zero resultados aparece a caixa "Nenhuma proposta encontrada com esses filtros", que não oferece criar.

Feito além do checklist

Dois placeholders para a listagem não terminar em 404. A tela tem linha clicável (`rowHref`) e botão de criar, e nenhum dos dois destinos existe ainda — são os blocos seguintes. Ficaram dois `page.tsx` mínimos com `<EmConstrucao />`: `propostas/nova` e `propostas/[id]`. O de criação já leva o **guard de escrita** (`redirect('/propostas')` para quem não é admin nem comercial), porque o `layout.tsx` de propostas libera `visualizador` também — a rota de criar não pode herdar essa permissão, e deixar o guard para depois é o tipo de coisa que não volta.

A triplicação de `computePeriodoCutoff` foi eliminada. A mesma função estava copiada em quatro lugares (`/orcamentos`, `/fd` e as duas rotas de export em `api/export`), e Propostas seria a quinta. Virou `src/lib/listagem.ts`, com `PERIODO_OPTIONS` junto — as opções do select também estavam duplicadas nos dois `filters.tsx`. Os quatro chamadores passaram a importar; nenhum mudou de comportamento, o parâmetro `hoje` é opcional e só existe para o teste poder fixar a data. `src/lib/listagem.ts` não tem `use client` nem import de React, então serve Server Component, Server Action e Client Component, igual aos outros helpers de `src/lib`.

Verificação

COMANDO	RESULTADO
<code>npm test</code>	31 casos, todos passando (eram 17)
<code>npx tsc --noEmit</code>	limpo

COMANDO	RESULTADO
<code>npm run lint</code>	sem warnings nem erros
<code>npm run build</code>	compila, 22 rotas (eram 19)

Os 14 casos novos cobrem o que a listagem tem de lógica pura: os seis de `obraIdsDaBusca` (nome, código, união com os clientes que casaram sem duplicar, busca vazia e sem match) e os oito de `listagem.ts` (os três cutoffs, o valor desconhecido da querystring virando "sem filtro", a garantia de não mutar a data recebida, e o `sanitizeBusca` removendo o que quebra o `.or()` sem estragar acento, hífen e barra).

Seguindo a lição do 4.1, o retorno do `select()` usa `as` e não `as unknown as`, para o `tsc` continuar comparando a string da query com o `Database` gerado — se uma coluna do select deixar de existir, o build acusa.

O que não foi verificado: nada rodou contra o banco. `npm run build` só prova que a query compila contra os types gerados de gc-dev; a listagem carregando dados de verdade, o JOIN aninhado `obra:obras(codigo_obra, nome, cliente:clientes(nome))` sob RLS e o comportamento do filtro de vencidas com dados reais continuam sem conferência. Falta a tela ser aberta com propostas cadastradas em gc-dev.

O que vem depois

O roteiro do 4.2 segue valendo, com o item 1 agora fechado:

- Listagem. Este bloco.**
- Criação e edição.** `proposta-form-helpers.ts` com schema zod e o par `formValuesToPayload` / `propostaToFormValues`, mais `proposta-form.tsx`. O form trabalha em 0 a 100 e o payload em fração (`pctToFraction` / `fractionToPct`), o schema reproduz `desconto <= valor_total` e chama `validarSomaPct`. Substitui o placeholder de `propostas/nova`.
- Detalhe e mudança de status.** `[id]/page.tsx`, `detail-header.tsx` (com o mesmo selo "Vencida" da listagem) e `status-dialog.tsx` sobre `canChangePropostaStatus`. Substitui o placeholder de `propostas/[id]`.
- Anexos**, sobre a coluna `anexos` jsonb e o bucket `anexos`.
- Permissão de escrita nas Server Actions** — o guard de layout protege a rota, não a ação.
- Exportação.** `api/export/propostas` mais o `ExportButton` na listagem. Ficou de fora aqui de propósito: o checklist do 4.3 não pede, e o botão sem endpoint só dá erro. Quando entrar, a rota já tem `computePeriodoCutoff` e `sanitizeBusca` compartilhados, e precisa repetir a resolução de IDs de `obraIdsDaBusca` para o filtro de busca bater com o da tela.
- Painel financeiro da proposta**, se entrar no escopo (view `propostas_financeiro`).

Pendências

1. **Listagem não verificada contra o banco.** Detalhe na seção de verificação. É a única pendência própria deste bloco.
2. **Herdada do 4.1, ainda aberta:** `supabase migration repair --status applied` nas dez versões antes de qualquer `db push`; depende de personal access token e `supabase link`. Junto seguem a revogação do token antigo, usado para gerar os types, e gc-prod quando despausar.

Estado do repositório

As mudanças estão **no working tree, sem commit**. Nada foi commitado nem enviado, conforme a regra de trabalho. O push para o GitHub continua bloqueado de qualquer forma: `brenocmlo` não tem permissão de escrita em `cremalet/gc-sistema`.

Duas coisas seguem fora deste trabalho, de propósito, porque não são do 4.3 e apagar seria destrutivo: `docs/4.1-status-entrega.{html,md,pdf}` e `package-lock.json` aparecem como deletados no working tree sem estar em stage, de antes deste bloco, e `supabase/criar_usuario_admin.sql` está untracked, também de antes.

Arquivos

ARQUIVO	O QUE É
<code>src/app/(app)/propostas/page.tsx</code>	reescrito. A listagem, sobre o select do 4.2
<code>src/app/(app)/propostas/filters.tsx</code>	novo. Busca, obra, status + vencidas, período
<code>src/app/(app)/propostas/propostas-table.tsx</code>	novo. As sete colunas e o selo "Vencida"
<code>src/app/(app)/propostas/nova/page.tsx</code>	novo. Placeholder com guard de escrita
<code>src/app/(app)/propostas/[id]/page.tsx</code>	novo. Placeholder do detalhe
<code>src/lib/listagem.ts</code>	novo. <code>computePeriodoCutoff</code> , <code>PERIODO_OPTIONS</code> , <code>sanitizeBusca</code>
<code>src/lib/listagem.test.ts</code>	novo. 8 casos
<code>src/lib/propostas.ts</code>	<code>obraIdsDaBusca</code> e <code>ObraBuscaRef</code>
<code>src/lib/propostas.test.ts</code>	6 casos novos, sobre a busca
<code>src/app/(app)/orcamentos/page.tsx</code> , <code>filters.tsx</code>	passam a usar <code>@/lib/listagem</code>
<code>src/app/(app)/fd/page.tsx</code> , <code>filters.tsx</code>	passam a usar <code>@/lib/listagem</code>

ARQUIVO	O QUE É
<code>src/app/api/export/{fd,orcamentos}/route.ts</code>	passam a usar <code>@/lib/listagem</code>
<code>src/components/StatusBadge.tsx</code>	inalterado, já tinha as cores pedidas
<code>docs/4.3-status-entrega.md</code>	este documento